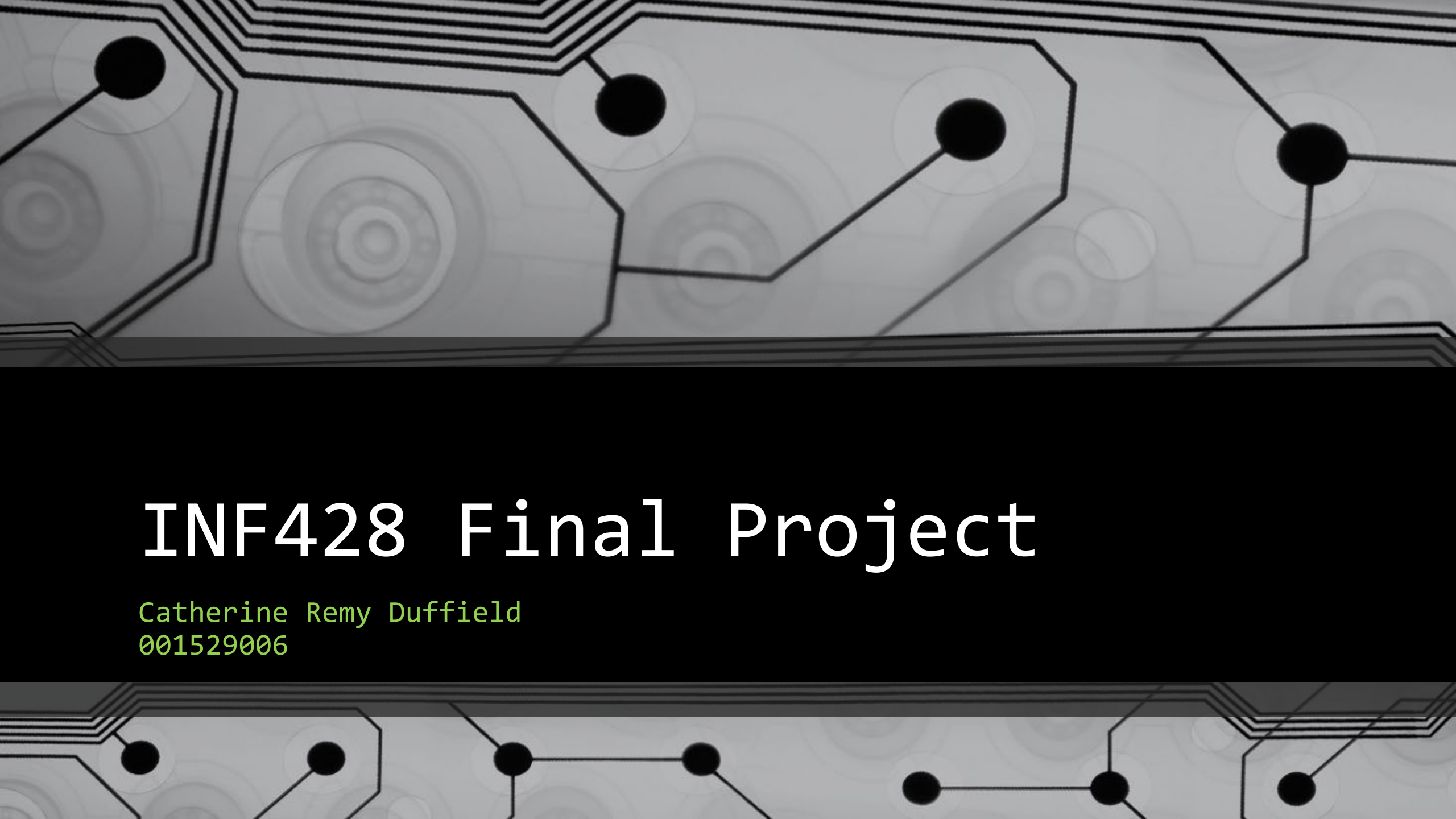


The background of the slide is a grayscale image of a circuit board. It features a complex network of black lines representing traces, with several large black circular pads or vias. The background has a subtle, repeating pattern of concentric circles, giving it a technical and digital feel.

INF428 Final Project

Catherine Remy Duffield
001529006

Service Payment Call Predictor

- Assignment Information
- Cleaning up Data
- Lessons Taught
- Model Aptitude
- Review

The Assignment

Background and information of the project

Project Call Predictor

This project aimed to create a Python model to predict the likelihood of policyholders with payments due to an insurance company making a service payment call. Identifying these policyholders will determine the best targets to send emails encouraging the use of automated payment channels instead. Data on customers with payment due in the next five days was provided. The primary tool to achieve the insurance company's endeavor was Python.

Data Cleaning

Preparing the data for use

Steps to Clean Data

- The data was assessed, and 0 was deemed an acceptable replacement for nulls. It can be assumed that the nulls represented accounts less than three months old.
- Two columns of the data could be simplified to numbers by removing the characters preceding the numbers. These characters were identical throughout the columns, with only the numbers indicating differences.

```
# fill null data
clean_df = df.fillna(0)
clean_df.info()
```

```
# simplifying data in columns
clean_df['POLICY_RATE'] = clean_df['RTD_ST_CD'].str.lstrip('ST_S').astype('int64')
clean_df['MART_No'] = clean_df['MART_STATUS'].str.lstrip('MS_S').astype('int64')
clean_df.drop(['RTD_ST_CD', 'MART_STATUS'], axis=1, inplace=True)
print(clean_df.head())
```

Continuing to Clean Data

```
# fix data type to numbers
clean_df.loc[clean_df['CustomerSegment'] == 'NONE', 'CustomerSegment'] = 0
clean_df.loc[clean_df['GENDER'] == 'M', 'GENDER'] = 0
clean_df.loc[clean_df['GENDER'] == 'F', 'GENDER'] = 1
clean_df[['CustomerSegment', 'GENDER']] = clean_df[['CustomerSegment', 'GENDER']].apply(pd.to_numeric)
print(clean_df.head())
```

- The last two columns, still containing strings, were CustomerSegment and GENDER. These columns required different changes but with similar coding. For CustomerSegment, the only non-numerical entry was NONE, replaced by 0, which was unused in that column. GENDER was smoothly changed to a binary with male, represented by M, equaling 0, and female, represented by F, equaling 1. An additional line of code was entered to ensure the two columns were identified as containing numeric data types.

Teaching Method

How to ensure the machine learns

Machine Learning Method

```
# import ml packages
from sklearn.model_selection import train_test_split as tts
from sklearn.linear_model import LogisticRegression as lr
from sklearn.metrics import classification_report as c_r
import sklearn.metrics as metrics
```

```
# select independent and dependent variables
X = clean_df[['CustomerSegment',
              'Tenure',
              'Age',
              'GENDER',
              'CHANNEL1_6M',
              'CHANNEL2_6M',
              'CHANNEL3_6M',
              'CHANNEL4_6M',
              'CHANNEL5_6M',
              'RECENT_PAYMENT',
              'PAYMENTS_6M',
              'NOT_DI_3M',
              'NOT_DI_6M',
              'EVENT2_90_SUM',
              'LOGINS',
              'MART_No']]
y = clean_df['Call_Flag']
```

- As one who is relatively new to coding learning models, the more familiar sklearn packages were chosen to obtain the prediction model.
- A total of 16 independent variables were selected to assist in predicting a possibility of a call, with identified results depicted in the Call_Flag column.

Further Machine Learning

The primary bulk of the learning was performed using logistic regression from sklearn with a randomized test size of 20%.

```
# data set split for train set and test set
x_train, x_test, y_train, y_test = tts(x, y, test_size=0.2, random_state=50)

# the lesson
call_lr = lr(solver='lbfgs', max_iter=500000).fit(x_train, y_train)

# the test
predict = call_lr.predict(x_test)
```

Evaluation

How functional is the model

	precision	recall	f1-score	support
0	0.97	1.00	0.98	25101
1	0.50	0.06	0.11	917
accuracy			0.96	26018
macro avg	0.73	0.53	0.55	26018
weighted avg	0.95	0.96	0.95	26018
Accuracy: 0.9647167345683757				
Recall: 0.06324972737186478				
Precision: 0.49572649572649574				

ML Report Card

An f1-score of 96% accuracy is acceptable for a beginner's attempt at creating a prediction model but would require more testing to consider it deliverable.

Take *Away*

Signing Off

- Coding is impressive, and knowing what I can achieve with it is inspiring. I spent so much time researching what could be done that I am excited to see more.
- This project still needs to be completed as I once again attempted more than I could handle by trying the extra undergrad credit. I hope to have time in the future to figure it out!